# PVLI

- lib/ bibliotecas y motor
- types/ index, phaser y matter (a estár)
- assets/
  - backgrounds (fondos varios)
  - characters (personajes)
  - UI (elementos de la interfaz)
  - audio (sonidos y efectos)
- src
  - game.js (es el arranque y config de phaser inicial)
  - ① scenes/ menuScene.js (menú ppal)
    combatScene.js (escena de combate)
    resultScene.js ?? no se si meterla como transición o hacer como con los QTE.
  - ② combat/ combat.js pts aura, turnos, resolución de acciones
    damage.js calcula el daño hecho/recibido.
    variants.js generación/renovación de variantes
    enemyIA.js intención y ataque enemigo
    effects.js estados, combos y efectos temporales
    bonuses.js favor, garrapon y carga de defensa

③ qte/

- miradaQTE — detener indicador
- porQTE — recordar direcciones
- carcassonedQTE — emparejar piezas y huecos
- improvisacionQTE — detectar elemento discordante
- gashapon — pulsar tecla a tiempo.

④ ui/

- combatHUD.js — info en pantalla (barras, cartas, intención, indicadores, etc...

⑤ data/

- gameData.js — definiciones y ajuste de valores (ataques, enemigos, balanceo, calculos)

— · —

① Scenes

- menuScenes.js — presenta el menú de inicio para dar comienzo
- combatScene.js — escena principal del combate.

(?) - resultScene.js — presenta el resultado del combate (o como los QTE?)

② combat/

combat.js: conserva la situación real (aura, turno, movimientos disponibles, intención, estados, bonificaciones...) recibe la acción o resultado y decide si el entrenamiento continua

damage.js: aquí va el daño en función de la calidad, los bonuses, las reducciones, etc...

variants.js: genera la oferta de movimientos disponibles y recibe sus características (dificultad, daño y estado).

enemyAI.js: genera intención de ataque, calidad y cuando hay ataque especial, así queda separado el comportamiento de jugador y enemigo.

effects.js: marcas de estado, bonos de estado, protección, cadencia de efectos, dificultad acumulada, etc...
Aquí están juntos todos los condiciones y reglas de los efectos activos.

bonuses.js: Favor del público, gashapon, cadena de perfects acumulada y carga defensiva disponible.

③ qte/ cada prueba por separado, cada una recibe los parámetros de la prueba y devuelve el resultado a la escena de combate.

- mirada QTE: recibe indicador y valora donde se detiene.
- pose QTE: muestra secuencia de pose y comprueba la entrada
- carcass real QTE: presenta el puzzle parcial y valora las combinaciones letra-número
- ✗ ✓ improvisación QTE: muestra la receta y comprueba el elemento discordante
- ✗ ✓ gashapon: muestra la fecha a pulsar, el tiempo disponible y chequea el resultado.

④ ui/

combatHUD.js construye las barras, el turno, las cartas, ejecución enemiga, marcas de estado, medidores, etc... todos los elementos visibles de la interfaz.

por ejemplo guardaremos los pts de aura en combat.js, aquí solo representamos, y se evita que haya conflicto por guardar dos valores potencialmente distintos.

⑤ data/

gameData.js: definiciones y valores ajustables: ataques, enemigos probabilidades, parámetros varios

Ej: los datos indican el daño del ataque y las probabilidades del rival: damage.js sabe calcular el resultado y enemyAI sabe seleccionar el ataque

combat.js, por ejemplo se encargaría de utilizar estos datos y definiciones para crear el combate y entregar a la escena los parametros de las pruebas

⑥ assets aquí orden que sino va a ser un caos.

backgrounds / los diferentes fondos / escenarios.
characters / sprites y animaciones de Paquito y los enemigos
ui / cartas, marcos, retratos, iconos, etc...
audio / música, efectos, cualquier recurso de audio.

Ej: un turno:

1. combatHUD.js: jugador elige Mirada y la interfaz comunica dicha elección.

2. combatScene.js llega la elección de combatHUD, se la entrega a combat.js y abre el QTE indicado.

3. MiradaQTE.js: ejecuta la prueba y devuelve resultado (p.ej. Perfecto)

4. combatScene.js: entrega el resultado al combate propio.

5. combat.js y auxiliares: calculan daño, resuelven efectos y actualizan estado.

6. combatScene.js y combatHUD.js: representan y anotan lo sucedido y actualizan la información, haciendo esa anotación visible.

7. combat.js y enemyAI.js: si el enfrentamiento continúa, se resuelve la intención, etc...